

Documento de Casos de Teste — Integração ViaCEP (Checkout Inteligente)

Módulo: tests/test_viacep.py
Arquivo de testes: tests/test_viacep.py
Framework: pytest + requests

Funções testadas

Função	Descrição
test_cep_valido_retorna_200_e_dados	Garante que um CEP válido traz os dados corretos de endereço
test_cep_inexistente_retorna_flag_de_erro	Garante o retorno da flag de erro para CEPs não encontrados
test_cep_formato_invalido_retorna_400	Verifica se a API barra requisições com formato inválido
test_cep_com_hifen_na_url_retorna_200	Verifica a resiliência da API ao receber CEPs com máscara
test_estrutura_do_json_retornado_schema	Garante que o contrato de chaves do JSON não foi modificado
test_tempo_de_resposta_da_api	Verifica se a API responde em tempo aceitável para o usuário

Casos de Teste

CT-01 — Consulta de CEP Válido e Existente

Campo	Detalhe
ID	CT-01
Função testada	test_cep_valido_retorna_200_e_dados
Objetivo	Validar se a API retorna os dados corretamente para preencher o formulário HTML
Entradas	cep = "01001000"
Resultado esperado	Status 200 e dados do endereço (Praça da Sé)
Técnica	Igualdade direta e ausência de chave (assert ==, assert not in)
Tipo	Caminho Feliz

```
def test_cep_valido_retorna_200_e_dados(api_client, base_url):
    cep = "01001000"
    url = f"{base_url}{cep}/json/"

    response = api_client.get(url)
    dados = response.json()

    assert response.status_code == 200
    assert "erro" not in dados
    assert dados["cep"] == "01001-000"
    assert dados["logradouro"] == "Praça da Sé"
    assert dados["bairro"] == "Sé"
    assert dados["localidade"] == "São Paulo"
```

CT-02 — CEP formatado corretamente, mas inexistente

Campo	Detalhe
ID	CT-02
Função testada	test_cep_inexistente_retorna_flag_de_erro
Objetivo	Validar se um CEP inexistente retorna a flag de erro para o Front-end interceptar
Entradas	cep = "99999999"
Resultado esperado	Status 200 e payload {"erro": "true"}
Técnica	Extração de chave (get()) e Igualdade direta
Tipo	Caminho Infeliz (Validação Semântica)

```
def test_cep_inexistente_retorna_flag_de_erro(api_client, base_url):
    cep = "99999999"
    url = f"{base_url}{cep}/json/"

    response = api_client.get(url)
    dados = response.json()

    assert response.status_code == 200
    assert dados.get("erro") == "true"
```

CT-03 — Validação de API para CEPs incompletos

Campo	Detalhe
ID	CT-03

Campo	Detalhe
Função testada	test_cep_formato_invalido_retorna_400
Objetivo	Validar que a API rejeita formatos de CEP inválidos no Back-end
Entradas	cep = "123"
Resultado esperado	Status 400 (Bad Request)
Técnica	Igualdade direta de código HTTP
Tipo	Caminho Infeliz (Validação de Sintaxe)

```
def test_cep_formato_invalido_retorna_400(api_client, base_url):
    cep_invalido = "123"
    url = f"{base_url}{cep_invalido}/json/"

    response = api_client.get(url)
    assert response.status_code == 400
```

CT-04 — API processando CEP enviado com Hífen

Campo	Detalhe
ID	CT-04
Função testada	test_cep_com_hifen_na_url_retorna_200
Objetivo	Validar a resiliência do back-end ao receber dados não higienizados
Entradas	cep = "01001-000"
Resultado esperado	Status 200 e dados do endereço
Técnica	Igualdade direta (assert ==)
Tipo	Caminho Alternativo

```
def test_cep_com_hifen_na_url_retorna_200(api_client, base_url):
    cep_com_hifen = "01001-000"
    url = f"{base_url}{cep_com_hifen}/json/"

    response = api_client.get(url)
    dados = response.json()

    assert response.status_code == 200
    assert "erro" not in dados
    assert dados["cep"] == "01001-000"
```

CT-05 — Verificação de Contrato (Schema JSON)

Campo	Detalhe
ID	CT-05
Função testada	test_estrutura_do_json_retornado_schema
Objetivo	Garantir que a API externa não alterou a estrutura das chaves do objeto
Entradas	cep = "01001000"
Resultado esperado	Todas as chaves obrigatórias presentes no dicionário retornado
Técnica	Laço de repetição e contenção (assert in)
Tipo	Validação Estrutural / Schema

```
def test_estrutura_do_json_retornado_schema(api_client, base_url):
    cep = "01001000"
    url = f"{base_url}{cep}/json/"

    response = api_client.get(url)
    dados = response.json()

    chaves_esperadas = [
        "cep", "logradouro", "complemento", "bairro",
        "localidade", "uf", "ibge", "gia", "ddd", "siafi"
    ]

    assert response.status_code == 200
    for chave in chaves_esperadas:
        assert chave in dados, f"A chave obrigatória '{chave}' sumiu do retorno!"
```

CT-06 — Desempenho e SLA de Resposta

Campo	Detalhe
ID	CT-06
Função testada	test_tempo_de_resposta_da_api
Objetivo	Validar se o tempo de resposta garante uma boa experiência ao usuário no front-end
Entradas	cep = "01001000"
Resultado esperado	Tempo de requisição menor que 2 segundos
Técnica	Comparador de limite (assert <)

Campo	Detalhe
Tipo	Teste de Performance / NFR (Requisito Não Funcional)
<pre>def test_tempo_de_resposta_da_api(api_client, base_url): cep = "01001000" url = f"{base_url}/{cep}/json/" response = api_client.get(url) tempo_resposta = response.elapsed.total_seconds() assert tempo_resposta < 2.0, f"A API demorou muito! Tempo: {tempo_resposta}s"</pre>	

Resumo das técnicas utilizadas

Técnica	Quando usar
<code>assert valor1 == valor2</code>	Comparação de valores exatos (status HTTP, retornos de texto e booleanos)
<code>assert chave in objeto</code>	Garantir que uma propriedade existe dentro da estrutura do JSON de resposta
<code>assert chave not in objeto</code>	Garantir que o JSON retornado em sucesso está livre de flags de falha
<code>assert tempo < limite, "msg"</code>	Verificações de limite com mensagens customizadas para debug fácil em testes de performance

Como executar os testes

A partir da raiz do projeto, instale os requisitos caso seja a primeira vez e execute o pytest:

```
# Instalar as bibliotecas necessárias
pip install pytest requests

# Rodar todos os testes com saída simplificada
pytest

# Rodar detalhadamente para visualizar o resultado de cada caso
pytest -v

# Rodar um teste de falha/performance específico para debug
pytest tests/test_viacep.py::test_tempo_de_resposta_da_api
```

Resultado esperado ao rodar pytest -v

```
tests/test_viacep.py::test_cep_valido_retorna_200_e_dados PASSED
tests/test_viacep.py::test_cep_inexistente_retorna_flag_de_erro PASSED
tests/test_viacep.py::test_cep_formato_invalido_retorna_400 PASSED
tests/test_viacep.py::test_cep_com_hifen_na_url_retorna_200 PASSED
tests/test_viacep.py::test_estrutura_do_json_retornado_schema PASSED
tests/test_viacep.py::test_tempo_de_resposta_da_api PASSED

===== 6 passed in 2.10s =====
```